

2. Concept of OOPS

Object-Oriented Programming (OOP) is a programming paradigm that organizes code using **objects** and **classes**. It helps in creating reusable, maintainable, and scalable applications.

1. Classes and Objects

What is a Class?

A class is a blueprint for creating objects.

Syntax

```
classStudent:  
    pass
```

What is an Object?

An object is an instance of a class.

```
classStudent:  
    pass  
  
s1=Student()  
print(type(s1))
```

Output:

```
<class'__main__.Student'>
```

2. Constructor (init)

The constructor initializes object attributes when an object is created.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s1 = Student("Rahul", 85)

print(s1.name)
print(s1.marks)
```

Output:

```
Rahul
85
```

3. Instance Variables vs Class Variables

Instance Variable

Unique for each object.

```
class Student:
    def __init__(self, name):
        self.name = name
```

Class Variable

Shared among all objects.

```
class Student:
    school = "ABC School"

    def __init__(self, name):
        self.name = name

s1 = Student("John")
```

```
s2=Student("David")
```

```
print(s1.school)
```

```
print(s2.school)
```

Output:

```
ABCSchool
```

```
ABCSchool
```

4. Inheritance

Inheritance allows one class to acquire properties of another class.

```
classAnimal:  
    def speak(self):  
        print("Animal speaks")
```

```
classDog(Animal):  
    pass
```

```
d=Dog()  
d.speak()
```

Output:

```
Animalspeaks
```

5. Polymorphism

Same method behaves differently.

```
classDog:  
    def sound(self):  
        print("Bark")
```

```
class Cat:
    def sound(self):
        print("Meow")

animals = [Dog(), Cat()]

for animal in animals:
    animal.sound()
```

Output:

```
Bark
Meow
```

6. Encapsulation

Encapsulation means hiding data from direct access.

```
class Bank:
    def __init__(self):
        self.__balance = 1000

    def show_balance(self):
        print(self.__balance)

b = Bank()
b.show_balance()
```

Output:

```
1000
```

Trying:

```
print(b.__balance)
```

Produces:

AttributeError

7. Abstraction

Showing only essential details and hiding implementation.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car Started")

c = Car()
c.start()
```

Interview Questions

Q1. Difference between Class and Object?

Class	Object
Blueprint	Instance
Logical entity	Physical entity
Created once	Multiple objects possible

Q2. What is self?

`self` refers to the current object.

```
classTest:
defdisplay(self):
print("Hello")
```

Q3. Difference between Instance and Class Variable?

Instance Variable	Class Variable
Object specific	Shared
Defined inside constructor	Defined inside class

Q4. What is Inheritance?

Acquiring properties and methods from parent class.

Q5. What is Polymorphism?

One interface, many implementations.

Practical Programs

Program 1: Create Student Class

```
classStudent:
def__init__(self,name,marks):
self.name=name
self.marks=marks

defresult(self):
ifself.marks>=40:
print("Pass")
else:
print("Fail")

s=Student("Rahul",70)

print("Name:",s.name)
```

```
print("Marks:",s.marks)
s.result()
```

Program 2: Employee Details

```
classEmployee:
def__init__(self,name,salary):
self.name=name
self.salary=salary

defdisplay(self):
print(self.name,self.salary)

e=Employee("John",50000)
e.display()
```

Program 3: Rectangle Area

```
classRectangle:
def__init__(self,length,width):
self.length=length
self.width=width

defarea(self):
returnself.length*self.width

r=Rectangle(10,5)
print(r.area())
```

Program 4: Circle Area

```
importmath
```

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

c = Circle(5)
print(c.area())
```

Program 5: Car Class

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def start(self):
        print(self.brand, "started")

c = Car("BMW")
c.start()
```

Program 6: Bank Account Simulation

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
```



```
def show_balance(self):  
    print("Balance:", self.balance)  
  
acc = BankAccount("Rahul", 5000)  
  
acc.deposit(1000)  
acc.withdraw(2000)  
  
acc.show_balance()
```

Program 7: Calculator Class

```
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
c = Calculator()  
print(c.add(5, 3))
```

Program 8: Inheritance Example

```
class Person:  
    def show(self):  
        print("Person")  
  
class Student(Person):  
    pass  
  
s = Student()  
s.show()
```

Program 9: Single Inheritance

```
classAnimal:
defeat(self):
print("Eating")

classDog(Animal):
defbark(self):
print("Barking")

d=Dog()
d.eat()
d.bark()
```

Program 10: Multilevel Inheritance

```
classGrandParent:
defshow1(self):
print("Grand Parent")

classParent(GrandParent):
pass

classChild(Parent):
pass

c=Child()
c.show1()
```

Program 11: Multiple Inheritance

```
classFather:
defskill1(self):
print("Driving")

classMother:
```

```
defskill2(self):  
    print("Cooking")  
  
classChild(Father,Mother):  
    pass  
  
c=Child()  
c.skill1()  
c.skill2()
```

Program 12: Method Overriding

```
classAnimal:  
    defsound(self):  
        print("Animal Sound")  
  
classDog(Animal):  
    defsound(self):  
        print("Bark")  
  
d=Dog()  
d.sound()
```

Program 13: Polymorphism

```
classDog:  
    defsound(self):  
        print("Bark")  
  
classCat:  
    defsound(self):  
        print("Meow")
```

```
for animal in [Dog(), Cat()]:  
    animal.sound()
```

Program 14: Encapsulation

```
class Bank:  
    def __init__(self):  
        self.__balance = 1000  
  
    def get_balance(self):  
        return self.__balance  
  
b = Bank()  
print(b.get_balance())
```

Program 15: Abstraction

```
from abc import ABC, abstractmethod  
  
class Shape(ABC):  
  
    @abstractmethod  
    def area(self):  
        pass  
  
class Square(Shape):  
    def area(self):  
        print("Area of Square")  
  
s = Square()  
s.area()
```

Program 16: Book Class

```
classBook:
def__init__(self,title):
self.title=title

defdisplay(self):
print(self.title)

b=Book("Python")
b.display()
```

Program 17: Product Class

```
classProduct:
def__init__(self,name,price):
self.name=name
self.price=price

defdiscount(self):
returnself.price*0.9

p=Product("Laptop",50000)
print(p.discount())
```

Program 18: Teacher Class

```
classTeacher:
def__init__(self,name,subject):
self.name=name
self.subject=subject

defdisplay(self):
print(self.name,self.subject)
```

```
t=Teacher("Amit","Math")
t.display()
```

Program 19: Inventory Class

```
classInventory:
    def__init__(self):
        self.items= []

    defadd_item(self,item):
        self.items.append(item)

i=Inventory()

i.add_item("Pen")
i.add_item("Book")

print(i.items)
```

Program 20: Library Management Mini Project

```
classLibrary:
    def__init__(self):
        self.books= []

    defadd_book(self,book):
        self.books.append(book)

    defissue_book(self,book):
        ifbookinself.books:
            self.books.remove(book)
            print("Book Issued")
        else:
```

```
print("Book Not Available")
```

```
def show_books(self):  
    print("Available Books:")  
    for book in self.books:  
        print(book)
```

```
lib = Library()
```

```
lib.add_book("Python")  
lib.add_book("Java")  
lib.add_book("C++")
```

```
lib.show_books()
```

```
lib.issue_book("Java")
```

```
lib.show_books()
```